Reproducing ReFT-Based Log Anomaly Detection (LogADReft) on HDFS and BGL

Deep Learning (AIC 401) — Assignment 02: Reproduction of Selected Research Paper

Muhammad Abdullah Abrar (01-134232-223) · Shehryar Zafar (01-134232-236) · BSCS 7A
Department of Computer Science, Bahria University Islamabad · Submitted to Ma'am Nabeela Bibi
Repository: https://github.com/mabdullahabrar/DeepLearning-Paper

Abstract

We reproduce the ReFT-based baselines of Lim, Zhu and Pang (2025) for log anomaly detection: RoBERTa-ReFT and GPT2-ReFT on HDFS and BGL, using the authors' released code and the preprocessed datasets of Le and Zhang (2022). Because of free-GPU limits we trained on only 2,700 sequences per run (plus 300 for validation) and tested on 3,000 sequences, so this is a scaled-down reproduction. We did not reproduce the published numbers. Our anomaly-class F1 was 0.73 (RoBERTa) and 0.76 (GPT-2) on HDFS against 0.9965 and 0.9971 in the paper, and 0.80 and 0.19 on BGL against 0.9558 and 0.8919; macro-averaged F1 was 0.85, 0.87, 0.89 and 0.56. The most likely contributor is the small training budget (0.59% of the HDFS and 3.6% of the BGL training sequences), but we did not test this by a larger run. We also document reproducibility problems: missing training details in the paper, a crash in the released data loader under its README configuration, a dependency version break, file-name mismatches and an unstated difference in metric averaging. The RoBERTa-ReFT runs are the reference baseline for Assignment 03.

1. Introduction and research objective

Log anomaly detection (LAD) flags abnormal log sequences that indicate failures or attacks in large systems. Lim et al. (2025) adapt pretrained language models to LAD with two parameter-efficient methods, LoRA and ReFT, and report results on four public datasets. Our objective is to implement and train their ReFT baselines for the two smallest backbones (RoBERTa and GPT-2) on HDFS and BGL, compare the results with the published ones, and record what was difficult or unclear. We do not reproduce Llama-3 (8B parameters), LoRA, Spirit, Thunderbird or the paper's additional experiments, because they exceed the compute available to us.

2. Dataset and preprocessing

Source. We use the parsed logs distributed by Le and Zhang (2022) on Zenodo (record 8115559, files preprocessed.zip and raw.zip). HDFS originates from Xu et al. (2009) and BGL from Oliner and Stearley (2007). Table 1 lists the dataset sizes as given in the dataset README.

Dataset	Size	# log lines	# anomalies	Anomaly ratio
HDFS	1.5 GB	11,175,629	16,838	2.93%
BGL	743 MB	4,747,963	348,460	7.34%

Table 1. Full datasets as listed in the Le & Zhang repository README (log-level counts).

Samples and classes. A sample is a sequence of parsed event templates with a binary label (normal or anomalous). The model input is the parsed event-template text, with the events of a sequence joined by spaces; variable parts of each message have already been replaced by the log parser's wildcard. HDFS sequences are session windows grouped by block ID. BGL sequences are fixed windows of 50 events (step 50). A sequence is anomalous if it contains an anomalous event. This follows the paper and the authors' data loader.

Split. The first 80% (chronological) is training data and the last 20% is test data. Following the authors' code, 10% of the sampled training sequences, stratified by class, are held out as a validation set used only for per-epoch monitoring. The test set is used only for the final evaluation. Table 2 shows our processed splits before subsampling.

Dataset	Split	Sequences	Anomalous	Anomaly %	Median tokens*	>510 tokens*
HDFS	train	460,048	13,505	2.94%	337	1.6%
HDFS	test	115,013	3,333	2.90%	337	1.8%
BGL	train	75,416	7,421	9.84%	252	21.0%
BGL	test	18,854	1,389	7.37%	802	58.6%

*Table 2. Processed sequence splits (results/dataset_stats.csv). *Token counts are from a random sample of 2,000 sequences per split with the RoBERTa tokenizer.*

Our subsample. The authors' code provides --max_n_train_example and --max_n_eval_example. We used 3,000 training sequences (2,700 for training and 300 for validation) and 3,000 test sequences. Their sampler keeps at most 10% anomalies, so each test sample has 300 anomalous sequences (10%), against 2.9% (HDFS) and 7.4% (BGL) in the full test splits. Our training data is only 0.59% of the HDFS and 3.6% of the BGL training sequences. This deviation is forced by free-GPU time and prevents any claim of exact reproduction.

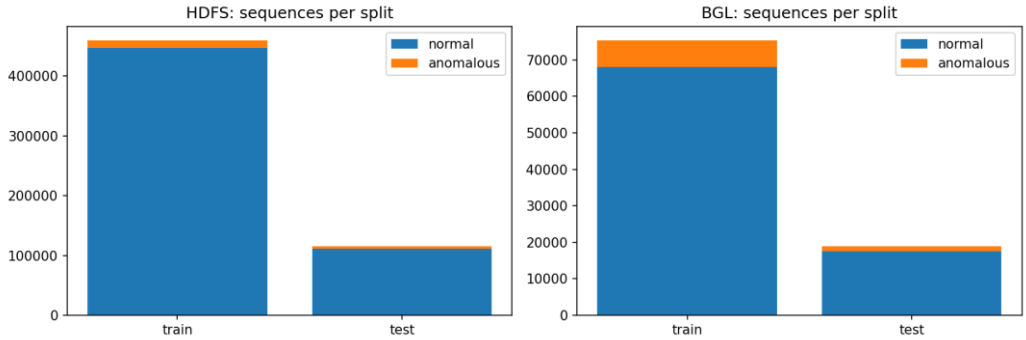


Figure 1. Normal and anomalous sequences per split (before subsampling).

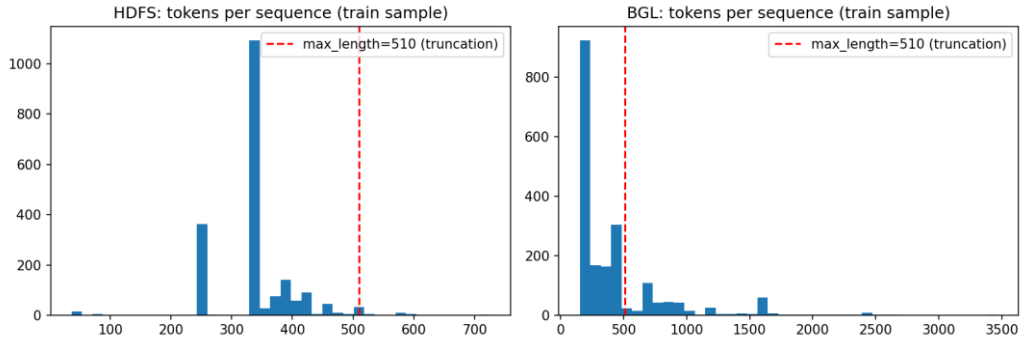


Figure 2. Tokens per training sequence. HDFS sequences mostly fit the 510-token limit; BGL sequences are often truncated, and BGL test sequences are much longer (median 802 tokens, 58.6% above 510) than training sequences (median 252, 21.0%).

3. Baseline model architecture

Each backbone is a pretrained transformer whose weights stay frozen. ReFT (LoReFT variant, Wu et al., 2024) learns a small intervention on hidden representations, $h \leftarrow h + R^T(Wh + b - Rh)$, where R has r = 8 orthonormal rows. The intervention is applied at one token position in every layer: the first token ([CLS]) for RoBERTa and the last token for GPT-2. That position feeds a classification head that outputs normal or anomalous, trained with cross-entropy loss. Both backbones have 12 layers, hidden size 768 and 12 attention heads (Hugging Face roberta-base: 124.6M parameters; gpt2: 124.4M; the paper lists GPT-2 as 117M). The trainable intervention parameters number 147,552 (0.12% of the model), which equals 12 layers × 12,296 (768×8 rotation plus a 768→8 projection with bias).

Setting	Value used
Intervention	LoReFT, rank 8, all layers (authors' scripts also pass --share_weights)
Position	first token (RoBERTa), last token (GPT-2)
Optimizer / learning rate	AdamW, 1e-4, linear schedule, 10% warmup
Regularization	dropout 0.1, weight decay 0
Batch size	32 effective (16 × 2 gradient accumulation on the T4)
Epochs	RoBERTa 3 (HDFS), 6 (BGL); GPT-2 6 (both)
Max sequence length / seed	510 tokens / 42
Trainable intervention parameters	147,552 for every run (excludes the classification head)
Classification head	RoBERTa: trained (--allow_cls_grad, about 592k parameters); GPT-2: not trained (flag not used in the authors' script)

Table 3. Configuration. Only the optimizer, learning rate, rank, position and batch sizes are stated in the paper; the rest comes from the authors' scripts and code.

Assumptions for details missing from the paper. Epochs per dataset, maximum length, dropout, warmup, learning-rate schedule, weight decay, seed, the validation split and the sampling rule are not in the paper. We took them from the authors' released scripts and code. For GPT-2 the script does not pass --allow_cls_grad, so its randomly initialised classification head stays frozen; we kept the authors' setting. We confirmed in main.py that only the RoBERTa head is unfrozen, and that the reported n_params counts the interventions only, even though a code comment says the head "will be counted".

4. Experimental setup

We ran the authors' main.py through a thin wrapper that changes nothing in training and only records anomaly-class metrics and the confusion matrix. Environment: Google Colab, Tesla T4 (15.6 GB), 13.6 GB RAM, 2 CPU cores, Python 3.13.16, PyTorch 2.11.0 (CUDA 13.0), transformers 4.45.1, pyreft 0.0.8, pyvene 0.1.6. Training and validation loss and validation metrics were logged every epoch (Appendix A); training and evaluation times are in Table 5. The configuration (args.json), logs and metrics of every run are saved in the repository. The trained ReFT weights were written to the Colab VM but were lost when the session was recycled before we downloaded them, so they are not included; rerunning the notebook regenerates them. We verified in the code that the test set is not used for model selection: validation is on a held-out part of the training sample, load_best_model_at_end is False and the early-stopping callback is commented out, so every run trains for the fixed number of epochs.

Deviations. (i) Subsampling, see Section 2. (ii) float32 and batch 16 × 2 gradient accumulation because the T4 has no bf16 support (the authors' code defaults to bf16 on CUDA). (iii) A single run with seed 42, as in the authors' scripts; the paper reports no variance.

5. Results and comparison with the original paper

The released evaluation code reports macro-averaged precision, recall and F1 for the ReFT classification path, while the paper does not say which averaging it uses. We therefore report both the macro scores and the anomaly-class (binary) scores. Original values are from Table 1 of Lim et al. (2025).

Dataset	Model	Metric	Paper	Reproduced (macro)	Diff.	Reproduced (anomaly class)	Diff.
HDFS	RoBERTa	F1	0.9965	0.8514	-0.145	0.7265	-0.270
HDFS	RoBERTa	Precision	0.9972	0.9637	-0.034	0.9721	-0.025
HDFS	RoBERTa	Recall	0.9958	0.7891	-0.207	0.5800	-0.416
HDFS	GPT-2	F1	0.9971	0.8666	-0.131	0.7628	-0.234
HDFS	GPT-2	Precision	0.9964	0.8383	-0.158	0.6940	-0.302
HDFS	GPT-2	Recall	0.9978	0.9026	-0.095	0.8467	-0.151
BGL	RoBERTa	F1	0.9558	0.8911	-0.065	0.8008	-0.155
BGL	RoBERTa	Precision	0.9693	0.9665	-0.003	0.9670	-0.002
BGL	RoBERTa	Recall	0.9433	0.8404	-0.103	0.6833	-0.260
BGL	GPT-2	F1	0.8919	0.5578	-0.334	0.1944	-0.698
BGL	GPT-2	Precision	0.8541	0.5658	-0.288	0.2213	-0.633
BGL	GPT-2	Recall	0.9417	0.5528	-0.389	0.1733	-0.768

Table 4. Original vs reproduced results (results/comparison_table.csv). Differences are reproduced minus paper.

Our results are well below the published ones, with one near match: the precision of RoBERTa-ReFT on BGL (0.967 against 0.969). Precision is high but recall is low for RoBERTa on both datasets: on HDFS it detects 174 of 300 anomalous test sequences (recall 0.58) with only 5 false alarms, and on BGL 205 of 300 (0.68) with 7 false alarms. GPT-2 behaves differently: on HDFS it finds more anomalies (254 of 300) with more false alarms (112), and on BGL it largely fails (52 of 300 detected, 183 false alarms, anomaly-class F1 0.19). All numbers come from one run per setting on a test sample with 300 anomalies, so a recall of 0.58 has roughly a ±0.06 binomial uncertainty at 95%, and the differences should be read as indicative.

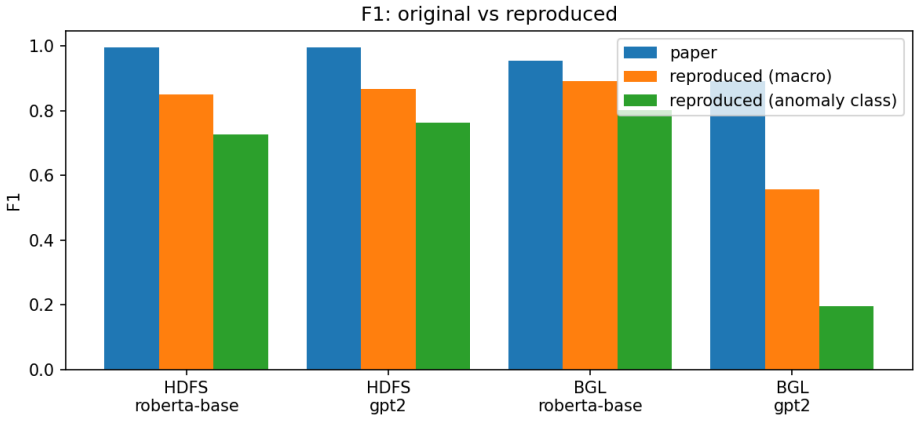


Figure 3. F1 of the paper versus our reproduction under the two averaging protocols.

Run	Train time (s)	Eval time (s)	TN	FP	FN	TP
RoBERTa / HDFS	549.5	81.2	2695	5	126	174
GPT-2 / HDFS	1300.9	91.1	2588	112	46	254
RoBERTa / BGL	1207.3	91.2	2693	7	95	205
GPT-2 / BGL	1402.7	98.6	2517	183	248	52

Table 5. Run times on a Tesla T4 and confusion matrices on the 3,000-sequence test samples (300 anomalous), from results/run_summary.csv.

6. Reproducibility challenges and discussion

What we found in the paper and the code

- **Missing details.** Epochs, maximum length, dropout, warmup, schedule, weight decay, seed, validation split and the subsampling rule are absent from the paper and had to be taken from the scripts.
- **Loader crash.** With the README's default loader settings (is_test_train_ratio=False and is_chronological=True) the sliding-window branch raises UnboundLocalError because n_train_for_n_test is only defined when is_test_train_ratio is True. We confirmed this in our own run (results/loader_bug_demo.txt). Using is_test_train_ratio=True gives the same chronological 80/20 split.
- **Documentation and packaging.** The README names the wrong script path (data_process/ instead of data_process_logs/); the loader writes 0.8train.pkl and 0.8test.pkl while the scripts read train.pkl and test.pkl; wandb, scikit-learn and pandas are imported but not listed in requirements.txt.
- **Dependency drift.** pyreft 0.0.8 only requires pyvene >= 0.1.4, so a fresh installation picks pyvene 0.1.8, whose changed internal format makes the authors' main.py fail with TypeError: 'LoreftIntervention' object is not subscriptable. Our first training attempt failed this way; pinning pyvene 0.1.6 (released the same day as pyreft 0.0.8) fixed it. Colab's Python 3.13 also differs from the Python 3.11 named in the README.
- **Metric averaging.** The ReFT classification evaluation uses macro averaging, the LoRA script uses default binary scores, and in-training validation uses binary F1. In the paper's Table 1 the F1 of GPT2-ReFT on BGL (0.8919) is lower than the harmonic mean of its reported precision and recall (0.8958), whereas the LoRA rows match exactly. This is consistent with different averaging for different rows, but it is not proof. In our runs the two protocols differ by 0.09–0.14 F1 (Table 4), so the choice matters.
- **Sampling changes the class ratio.** The authors' sampler caps anomalies at 10% of the test sample, so our precision (and anomaly-class F1) is not comparable with numbers from the full test splits.
- **Parameter counting.** The reported n_params is identical for RoBERTa and GPT-2 (147,552) because it counts only the interventions; the trained RoBERTa head (about 592k parameters) is not included, despite the code comment.

Answers to the assignment questions

Was the baseline implemented successfully? Partly. We ran the authors' implementation end to end for all four settings with the intended architecture (12 interventions, 147,552 trainable intervention parameters), and the validation F1 improved over the epochs (Appendix A). However, we trained on a small fraction of the data and our results do not match the paper, so we describe this as a scaled-down reproduction and not a successful replication of the published numbers.

How do the results compare, and why do they differ? Table 4 shows lower scores in every row except one near match (BGL RoBERTa precision). Our evidence points to three likely causes, none of which we isolated experimentally. (1) Training budget: we used 2,700 training sequences (0.59% of HDFS, 3.6% of BGL). RoBERTa on HDFS was still improving when training stopped (validation F1 0.00, 0.46, 0.72 after epochs 1 to 3), and its test F1 (0.73) matched the final validation F1 (0.72), which fits under-training. (2) Chronological shift and truncation on BGL: test sequences are much longer than training ones (median 802 vs 252 tokens; 58.6% vs 21.0% above the 510-token limit, Table 2). GPT-2 reads the last token, so truncation changes what it sees, and its validation F1 of 0.75 (random held-out part of the training period) fell to 0.19 on the chronological test set; RoBERTa, which uses the first token, fell less. (3) Setup differences: the frozen GPT-2 classification head, a single seed, the

10% anomaly sample, and the unstated metric averaging. A run with a larger sample and several seeds is needed to separate these causes.

Which details were hard or unclear? The missing hyperparameters, the metric averaging, whether the GPT-2 head is trained (it is not, in the released script), and how n_params is counted, as listed above.

Computational challenges. On a free T4 (no bf16, 15.6 GB) we had to use float32 and 16 × 2 gradient accumulation. Training took 549 s (RoBERTa, HDFS) to 1,403 s (GPT-2, BGL); the four runs took about 83 minutes in total including evaluation. Full splits (460,048 HDFS training sequences) were not feasible, which forced the subsampling. We also lost time to the pyvene version break and to Colab session restarts, which reset the working directory.

Remaining limitations. Subsampled data, a single seed, no confidence intervals, no Llama-3, no LoRA, only two of four datasets, a test sample with a different class ratio, and an unknown averaging protocol in the paper.

Use in Assignment 03. We will use RoBERTa-ReFT as the reference baseline, since GPT-2 on BGL is unstable in our setting. Any improvement must be compared with these numbers under the same samples and protocol, reporting both averaging schemes. If time allows, we will rerun the baseline with a larger training sample and several seeds so the comparison is not dominated by noise.

7. Conclusion and next steps

We trained and evaluated the ReFT baselines of Lim et al. (2025) with RoBERTa and GPT-2 on HDFS and BGL using the authors' code, and did not reproduce the published scores: RoBERTa reached an anomaly-class F1 of 0.73 (HDFS) and 0.80 (BGL), GPT-2 reached 0.76 and 0.19. The small training budget is the most likely cause, supported by the still-rising validation curve, but untested. In Assignment 01 we proposed log normalization and multi-source training to improve cross-dataset generalization. While reading the code we found that the model input is already the parsed event template, so numbers, IDs and similar variable parts are already masked by the parser. Our normalization idea must therefore target template wording and vocabulary differences between systems, not raw variables, and we will revise it for Assignment 03. Multi-source training is unaffected. Our BGL results also show that sequence length and truncation under chronological drift deserve attention in that assignment.

References

[1] Lim, Y. F., Zhu, J., Pang, G.: Adapting Large Language Models for Parameter-Efficient Log Anomaly Detection. PAKDD 2025, LNCS vol. 15876, Springer (2025). https://doi.org/10.1007/978-981-96-8298-0_26

[2] Lim, Y. F., Zhu, J., Pang, G.: LogADReft source code. https://github.com/mala-lab/LogADReft

[3] Le, V.-H., Zhang, H.: Log-based anomaly detection with deep learning: How far are we? ICSE 2022, pp. 1356–1367. Data: https://zenodo.org/record/8115559

[4] Wu, Z., Arora, A., Wang, Z., Geiger, A., Jurafsky, D., Manning, C. D., Potts, C.: ReFT: Representation finetuning for language models (2024).

[5] Xu, W., Huang, L., Fox, A., Patterson, D., Jordan, M. I.: Detecting large-scale system problems by mining console logs. SOSP 2009.

[6] Oliner, A. J., Stearley, J.: What supercomputers say: A study of five system logs. DSN 2007.

[7] Liu, Y., et al.: RoBERTa: A robustly optimized BERT pretraining approach (2019).

[8] Radford, A., et al.: Language models are unsupervised multitask learners. OpenAI blog (2019).

[9] Wolf, T., et al.: Transformers: State-of-the-art natural language processing. EMNLP 2020 (System Demonstrations).

Appendix A: training curves

Training loss and validation loss per epoch, and validation F1 (anomaly class, 300 validation sequences with 30 anomalies) per epoch. Validation sets are small, so the curves are noisy.

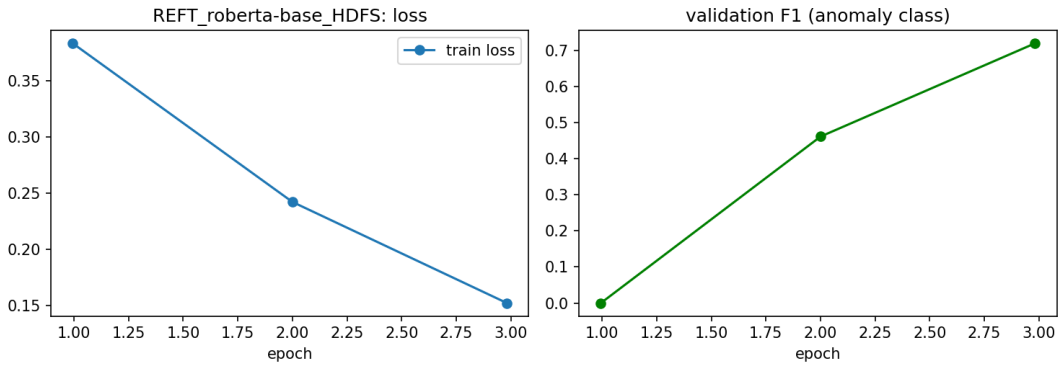


Figure A1. RoBERTa-ReFT on HDFS.

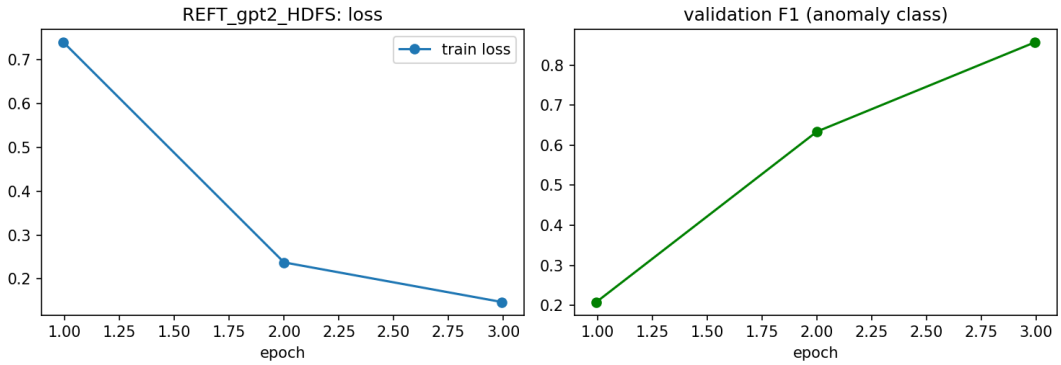


Figure A2. GPT2-ReFT on HDFS.

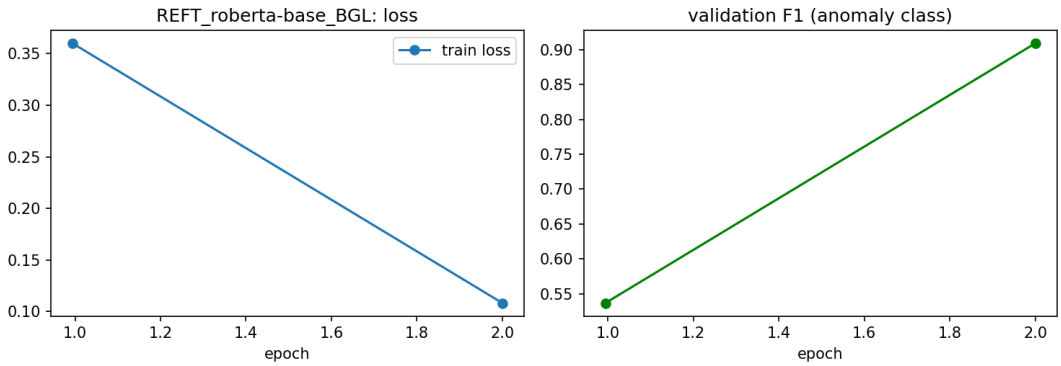


Figure A3. RoBERTa-ReFT on BGL.

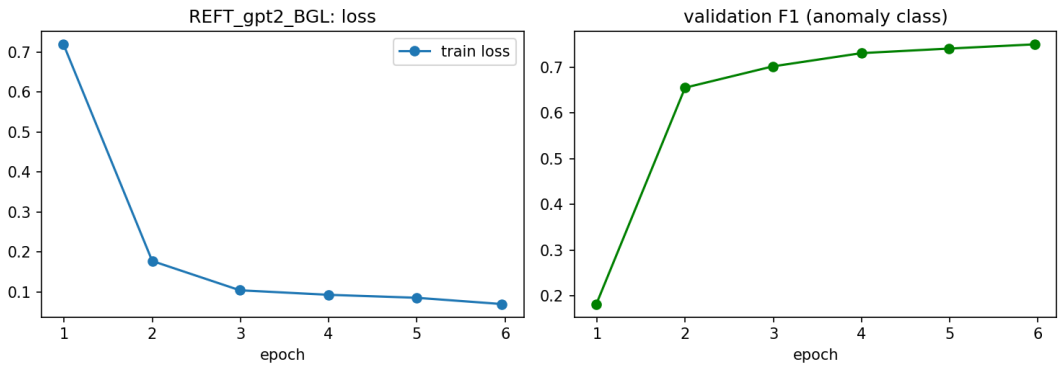


Figure A4. GPT2-ReFT on BGL.